

TEST PLAN

MODULE: SELL COURSE

Integrated Coding Education & Competitive Programming Platform

Field	Detail
Project Name	Integrated Coding Education & Competitive Programming Platform
Document Title	TEST PLAN – Sell Course Module
Version	v1.0
Prepared By	[QA Tester – Sell Course]
Reviewed By	QA Lead / Project Manager
Approved By	Stakeholder / Customer
Date	18/05/2026
Status	Draft

REVISION HISTORY

Version	Date	Author	Description
1.0	18/05/2026	[QA Tester]	Initial version – Sell Course module

1. INTRODUCTION

1.1 Purpose

- This Test Plan defines the testing approach, scope, schedule, resources, tools, and activities for the Sell Course module of the Integrated Coding Education & Competitive Programming Platform.
- The objective is to verify that all course-selling functions—including course listing, purchase flow, payment processing, enrollment, and access management—operate correctly and meet defined business requirements before production release.

1.2 System Overview

- The platform provides online coding education combined with competitive programming features.
- The Sell Course module allows instructors and administrators to publish courses for sale and enables students to browse, purchase, and access paid course content.

2. PROJECT OVERVIEW

The Sell Course module is a key revenue-generating feature of the platform. It encompasses the complete lifecycle from course creation and pricing configuration to student purchase, payment verification, and post-purchase access control.

Component	Technology / Detail
Frontend	React.js / Vue 3
Backend	Node.js / .NET API
Database	PostgreSQL / MongoDB
Authentication	JWT (Role-based: Admin, Instructor, Student)
Payment Gateway	VNPay / Stripe / PayPal (integration)
Deployment	Docker, CI/CD via GitHub Actions

3. TEST OBJECTIVES

- Verify all functional requirements of the Sell Course module are correctly implemented.
- Validate the end-to-end purchase flow from course browsing to successful enrollment.
- Ensure payment gateway integration processes transactions accurately and securely.
- Verify role-based access control: only paying students can access purchased course content.
- Detect and report defects early to reduce rework cost before production release.
- Ensure system stability under normal and peak load conditions.
- Validate data integrity: orders, invoices, and enrollment records are correctly stored.
- Verify error handling for failed payments, expired sessions, and invalid inputs.

4. TEST SCOPE

4.1 In Scope

Module / Feature	Description
Course Listing	Browse all available courses, filter by category/level/price, search
Course Detail Page	View course info, syllabus, instructor, pricing, reviews
Shopping Cart	Add/remove courses, apply discount/coupon codes
Checkout & Payment	Payment via VNPay/Stripe, order confirmation, invoice generation
Enrollment Management	Auto-enroll after payment, grant content access, enrollment records
Course Pricing Config	Admin/Instructor sets price, discount %, free/paid toggle
Order History	Student views past purchases, download invoices
Refund / Cancellation	Refund policy enforcement, partial/full refund workflow
Access Control	Restrict content to enrolled/paid students only
Notification	Email confirmation after successful purchase

4.2 Out of Scope

- Mobile application (iOS / Android) – separate test cycle.
- Third-party LMS integration (external course import).
- AI-based course recommendation engine.
- Live streaming / interactive session features.
- Admin financial reporting dashboard (covered by separate Finance module test plan).

5. TEST ITEMS

Item	Description	Type
Course Listing API	GET /api/courses – returns paginated course list	API
Course Detail API	GET /api/courses/{id} – full course data	API
Cart API	POST/DELETE /api/cart – manage cart items	API

Item	Description	Type
Order API	POST /api/orders – create purchase order	API
Payment API	POST /api/payment – initiate & confirm payment	API
Enrollment API	POST /api/enrollments – auto-enroll on payment success	API
Course Listing UI	Browse/search/filter screen	UI
Checkout UI	Cart & payment form screens	UI
Order History UI	Student order history & invoice download	UI
DB – orders table	Validate order/invoice records in PostgreSQL	Database
DB – enrollments table	Validate enrollment records post-payment	Database
Email Notification	Purchase confirmation email delivery	Integration

6. TEST TYPES

Test Type	Purpose	Tool
Functional Testing	Verify all sell-course business functions work correctly	Manual / Postman
Integration Testing	Verify payment gateway, email, enrollment API interactions	Postman / Newman
System Testing	End-to-end purchase flow validation	Manual
Regression Testing	Ensure new changes do not break existing sell-course features	Selenium / Playwright
Performance Testing	Evaluate course listing & checkout response under load	JMeter
Security Testing	Validate access control, prevent unauthorized access	OWASP ZAP / Manual

Test Type	Purpose	Tool
	to paid content	
Usability Testing	Verify intuitive checkout and course browsing experience	Manual
Compatibility Testing	Chrome, Edge, Firefox, Safari – desktop and mobile browser	BrowserStack
UAT	End-user acceptance of complete purchase flow	Manual

7. TEST APPROACH / TEST STRATEGY

7.1 Testing Levels

Level	Approach	Owner
Unit Testing	Dev-side unit tests for payment calculation, cart logic	Developer
Integration Testing	API tests covering payment gateway & enrollment triggers	QA Tester
System Testing	Full E2E test: browse → cart → pay → enroll → access	QA Tester
UAT	Customer/student validates the complete purchase experience	Customer / PM

7.2 Techniques

- **Equivalence Partitioning** – e.g., valid price ranges, coupon code categories (valid / expired / invalid).
- **Boundary Value Analysis** – e.g., minimum payment amount (1 VND), maximum cart items (10).
- **Decision Table Testing** – e.g., payment success/failure combinations with enrollment trigger.
- **Use Case Testing** – end-to-end scenarios: Guest browse → Register → Purchase → Access Course.
- **State Transition Testing** – Order states: Draft → Pending Payment → Paid → Enrolled → Refunded.
- **Risk-Based Testing** – prioritise critical payment and access-control test cases first.

8. ENTRY CRITERIA

- Functional requirements for Sell Course module are reviewed and approved.
- Test environment is set up with all required services (API, DB, payment sandbox).
- Build successfully deployed to QA environment with no critical build errors.
- Test data (student accounts, course data, coupon codes, payment sandbox credentials) prepared.
- All test cases for Sell Course module are written and peer-reviewed.
- Access to payment gateway sandbox (VNPay / Stripe test mode) is confirmed.

9. EXIT CRITERIA

- 100% of critical (Severity 1 & 2) test cases executed.
- No open Severity 1 (Critical) defects; Severity 2 defects have accepted workarounds.
- Pass rate $\geq 95\%$ for all Sell Course test cases.
- Performance: course listing API responds $< 2s$; checkout API responds $< 3s$ under 100 concurrent users.
- Test Summary Report completed and submitted to QA Lead.
- Customer UAT sign-off obtained.

10. TEST ENVIRONMENT

Component	Configuration
Frontend	React.js / Vue 3 + Vite, hosted on QA server
Backend API	Node.js / .NET 9 Web API – QA environment
Database	PostgreSQL 16 – dedicated QA instance
Payment Sandbox	VNPay sandbox + Stripe test mode
Email Service	Mailtrap (email testing sandbox)
Browsers	Chrome 124+, Edge 124+, Firefox 125+, Safari 17+
OS	Windows 11 (primary), macOS 14 (secondary)
Network	Internal QA network; VPN for remote testers

11. TEST TOOLS

Tool	Purpose	Used By
Jira	Defect tracking & test management	QA Team
Postman / Newman	API testing & automated API regression	QA Tester
Selenium / Playwright	UI automation & regression testing	QA Tester
JMeter	Performance & load testing on APIs	QA Tester
OWASP ZAP	Security vulnerability scanning	QA / Dev
Mailtrap	Email notification testing	QA Tester
GitHub Actions	CI/CD – auto-run Postman/Playwright on deploy	DevOps
BrowserStack	Cross-browser compatibility testing	QA Tester

12. TEST DATA MANAGEMENT

All test data will be isolated in the QA environment and will not affect production data.

Dataset	Purpose	Source
Valid Student Accounts	Positive login & purchase flow testing	Generated / Seeded
Invalid / Unverified Accounts	Negative testing – unauthorised purchase attempts	Generated
Course Catalogue (10+ courses)	Browse, filter, search, purchase tests	Seeded in QA DB
Valid Coupon Codes	Discount application positive testing	Generated via Admin
Expired / Invalid Coupons	Negative coupon testing	Generated
Payment Sandbox Credentials	Successful and failed payment scenarios	VNPay / Stripe sandbox
Large Dataset (1000+ courses)	Performance & pagination testing	Script-generated

13. ROLES AND RESPONSIBILITIES

Role	Responsibility
QA Lead	Manage overall test plan, review test cases, approve test reports
QA Tester (Me)	Write & execute Sell Course test cases, log defects, regression testing
Developer	Fix reported defects, support test environment setup
Project Manager	Approve test deliverables, coordinate UAT schedule
Customer / Student	Perform UAT on purchase & enrollment flows
DevOps	Maintain QA environment, CI/CD pipeline, payment sandbox config

14. TEST SCHEDULE

Phase	Start Date	End Date	Owner	Duration
Test Planning & Review	18/05/2026	19/05/2026	QA Tester	2 days
Test Case Design	20/05/2026	22/05/2026	QA Tester	3 days
Environment Setup	20/05/2026	21/05/2026	DevOps	2 days
Test Execution – Functional	23/05/2026	28/05/2026	QA Tester	5 days
Integration & Payment Tests	26/05/2026	28/05/2026	QA Tester	3 days
Performance & Security Tests	29/05/2026	30/05/2026	QA Tester	2 days
Regression Testing	31/05/2026	03/06/2026	QA Tester	3 days
UAT	04/06/2026	06/06/2026	Customer / PM	3 days
Test Closure & Report	07/06/2026	08/06/2026	QA Lead	2 days

15. DEFECT MANAGEMENT PROCESS

15.1 Workflow

New → Assigned → In Progress → Fixed → Retest → Closed / Reopened

15.2 Severity Levels

Severity	Level	Sell Course Example
Critical	S1	Payment processed but student not enrolled; double charge
High	S2	Checkout page crashes; coupon code never applies discount
Medium	S3	Invoice PDF missing course name; wrong currency displayed
Low	S4	Minor UI misalignment on course listing page; typo in button label

16. RISKS AND MITIGATION

Risk	Impact	Mitigation Plan
Payment sandbox instability	High	Prepare backup sandbox credentials; schedule re-test slots
Late API delivery by Dev	High	Parallel test case design; mock APIs with Postman Mock Server
Incomplete payment gateway docs	Medium	Contact gateway vendor; use official sandbox documentation
Insufficient test data	Medium	Prepare data generation scripts in advance
Scope creep (new payment	Medium	Change request process;

Risk	Impact	Mitigation Plan
methods)		update test plan before adding tests
QA environment downtime	Medium	Dedicated QA environment; DevOps on-call for quick restore

17. TEST DELIVERABLES

- Test Plan (this document) – v1.0
- Test Cases – Sell Course module (Jira / Excel)
- API Test Collection – Postman collection for all Sell Course APIs
- Defect Reports – logged in Jira with screenshots and reproduction steps
- Test Execution Logs – daily execution status
- Performance Test Report – JMeter results for course listing & checkout
- Test Summary Report – final sign-off document
- RTM (Requirement Traceability Matrix) – maps requirements to test cases

18. COMMUNICATION PLAN

Activity	Frequency	Participants
Daily QA Standup	Daily	QA Team, Dev Team
Defect Review Meeting	2x per week	QA Tester, Developers, QA Lead
Test Status Report	Weekly	QA Lead, Project Manager, Stakeholders
UAT Walkthrough	Once (UAT phase)	QA Tester, Customer, Project Manager
Test Closure Report	End of cycle	All stakeholders

19. APPROVALS

Name	Role	Signature	Date
	QA Tester (Sell Course)		
	QA Lead		

Name	Role	Signature	Date
	Project Manager		
	Customer Representative		

APPENDIX A – TEST CASE TEMPLATE

Field	Description
Test Case ID	SC-TC-001 (SC = Sell Course prefix)
Module	e.g., Checkout, Cart, Enrollment
Priority	High / Medium / Low
Preconditions	e.g., Student logged in; course exists with price VND 299,000
Test Steps	Step-by-step user actions
Expected Result	Expected system behaviour
Actual Result	Actual observed result during execution
Status	Pass / Fail / Blocked

Sample Test Cases

TC ID	Test Case Name	Precondition	Expected Result	Priority
SC-TC-001	Purchase paid course with valid payment	Student logged in, cart has 1 paid course	Order created, enrolled, confirmation email sent	High
SC-TC-002	Apply valid coupon code at checkout	Valid coupon in system	Discount applied, total price updated	High
SC-TC-003	Payment fails – insufficient funds	Sandbox card set to decline	Error message shown; student NOT enrolled	High
SC-TC-004	Non-enrolled student accesses paid content	Course not purchased	Access denied; redirected to purchase page	Critical
SC-TC-005	Add course to cart and remove	Student logged in	Cart updated; total recalculated	Medium

TC ID	Test Case Name	Precondition	Expected Result	Priority
	it			
SC-TC-006	View order history after purchase	Completed purchase exists	Order appears with date, amount, course name	Medium
SC-TC-007	Download invoice PDF	Order completed	PDF generated with correct data	Medium
SC-TC-008	Purchase free course	Free course exists	Enrolled immediately with no payment required	High

APPENDIX B – DEFECT REPORT TEMPLATE

Field	Description
Defect ID	SC-BUG-001
Summary	Short description of the defect
Module	Sell Course – e.g., Checkout
Steps to Reproduce	Numbered steps to reproduce the issue
Expected Result	What should happen
Actual Result	What actually happened
Severity	Critical / High / Medium / Low
Priority	P1 / P2 / P3
Environment	QA – Browser, OS, Build version
Attachment	Screenshot / video / log file
Status	Open / Assigned / Fixed / Retested / Closed

APPENDIX C – DECISION TABLE: PAYMENT & ENROLLMENT

Condition / Action	TC1	TC2	TC3	TC4
Valid Student	Y	Y	Y	N

Condition / Action	TC1	TC2	TC3	TC4
Account				
Valid Payment Card	Y	N	Y	–
Course in Cart	Y	Y	N	Y
Expected Result	Enrolled ✓	Payment Failed	Cart Error	Auth Error

APPENDIX D – USE CASE TESTING: PURCHASE A PAID COURSE

Step	User Action	Expected Result
1	Open Course Listing page	Course catalogue displayed with filters
2	Search for 'Python for Beginners'	Matching course appears in results
3	Click on course → View Detail	Detail page: price, syllabus, instructor shown
4	Click 'Add to Cart'	Course added; cart counter updated
5	Open Cart → Apply coupon 'SAVE10'	10% discount applied; total updated
6	Click 'Checkout'	Payment form displayed with order summary
7	Enter valid card details → Click 'Pay'	Payment processing spinner shown
8	Payment gateway returns success	Order confirmed; student auto-enrolled
9	Navigate to 'My Courses'	Purchased course appears in student dashboard
10	Check email inbox	Confirmation email received with invoice

END OF DOCUMENT